



Department of CSE, PES University

UE24CS341A: SOFTWARE ENGINEERING

5TH SEMESTER, ACADEMIC YEAR 2026-27

SOFTWARE ARCHITECTURE AND DESIGN SPECIFICATION

DATE: 04-10-2026

Software Architecture and Design Specification for Web Based ML Model Evaluator

Project: Web Based ML Model Evaluator

Version: 1.2

Authors: PES2UG24CS368 – PRATHAM KUMAR - prathamkumar041104@gmail.com
PES2UG24CS324 – NIRUPAMA JAYARAMAN - jnirupama06@gmail.com
PES2UG25CS824 – SHREYAS BELLAD - shreyasbellad2006@gmail.com
PES2UG24CS339 – PARINITHA ACHARYA - parinitha.acharya2006@gmail.com

Date: 04-10-2026

Status: Draft / Submitted

Revision History

Version	Date	Author	Change Summary	Approval
1.0	06-09-2026	Project Team	Initial Software Requirements Specification (SRS) document defining the functional and non-functional requirements, system scope, features, constraints, UML use cases, and requirements traceability for the Web Based ML Model Evaluator.	—
1.1	16-09-2026	Project Team	Software Test Plan (STP) prepared based on the SRS, defining the testing scope, test strategy, test environment, test schedule, test deliverables, risks, traceability, and test metrics.	—
1.2	04-10-2026	Project Team	Software Architecture and Design Specification (SAD) prepared based on the SRS, STP, and faculty-provided SAD template, defining the system architecture, components, design, interfaces, security architecture, UML diagrams, technology stack, and design decisions.	—

Approvals

Role	Name	Signature / Email	Date
Course Coordinator	Sheela Devi M.	sheelam@pes.edu	—

Table of Contents

S. No.	Table of Contents	Page No.
1	Introduction	2 – 4
2	Document Overview	5 – 6
3	Architecture	7 – 20
4	Design	21 – 38
5	Appendices	39 – 40

1. Introduction

1.1 Purpose

The purpose of this Software Architecture and Design Specification (SAD) is to define the architecture and detailed design of the **Web Based ML Model Evaluator**. This document translates the requirements specified in the Software Requirements Specification (SRS) into a structured architectural and design solution that can guide system implementation, integration, testing, and maintenance.

The SAD defines the major system components, their responsibilities and interactions, the selected architectural pattern and its rationale, technology stack and data-storage decisions, system risks and mitigations, requirements traceability, and security architecture. It also specifies key design aspects including system workflows, UML diagrams, API interfaces, error handling, logging and monitoring, user-experience considerations, and future enhancements.

The document serves as the architectural baseline for the development team and provides a consistent reference for understanding how the functional and non-functional requirements of the Web Based ML Model Evaluator are realized through the proposed system architecture and design. It is intended to remain consistent with the SRS, Software Test Plan (STP), and Requirements Traceability Matrix (RTM) established in the previous project phases.

1.2 Scope

The **Web Based ML Model Evaluator** is a web-based application designed to provide an integrated platform for evaluating and comparing supervised machine learning models using tabular datasets. The system architecture and design cover the complete workflow from user authentication and project management to dataset preparation, model training, evaluation, comparison, and report generation.

The scope of the system includes:

- User registration, authentication, and secure access to the application.
- Creation, viewing, renaming, and management of machine learning evaluation projects.
- Uploading and validation of CSV datasets.
- Previewing uploaded datasets, including relevant dataset information and basic analysis.
- Configuration of data preprocessing, including missing-value treatment, categorical-data encoding, feature scaling, and train-test splitting.
- Selection of the target column and classification or regression task.
- Selection and configuration of compatible machine learning models.
- Training and testing of selected models using held-out test data.
- Calculation of appropriate evaluation metrics for classification and regression tasks.
- Comparison and visualization of model performance.

- Selection of a preferred model based on evaluation results.
- Viewing previous evaluation results and maintaining evaluation history.
- Generation and downloading of evaluation reports.
- Secure storage and controlled access to user, project, dataset, model-run, result, and report information.
- Definition of architectural components, interfaces, data flow, security controls, and design decisions required to support the above functionality.

The initial scope is limited to **tabular supervised-learning tasks involving classification and regression**. Deep-learning models, image and video processing, real-time evaluation, and production deployment of trained models are excluded from the initial scope. The architecture is designed to maintain clear module boundaries so that additional capabilities can be incorporated in future versions without affecting the core system.

1.3 Audience

This Software Architecture and Design Specification (SAD) is intended for the following stakeholders involved in the development, evaluation, testing, and future maintenance of the **Web Based ML Model Evaluator**:

- **Project Developers and ML Engineers:** To understand the system architecture, component responsibilities, interfaces, data flow, machine learning processing modules, technology decisions, and design considerations required for implementation and maintenance.
- **QA Engineers and Test Coordinators:** To understand the architectural components, system interfaces, workflows, and design decisions that support integration, system, security, performance, and other testing activities defined in the Software Test Plan (STP).
- **Project Guide and Course Coordinator:** To review and evaluate the architectural decisions, system design, requirement traceability, security considerations, UML diagrams, and overall consistency of the project with the defined requirements.
- **Academic Users and Future Maintainers:** To understand the structure, functionality, architectural organization, and design of the system for academic use, maintenance, modification, and potential future enhancement.

This document therefore serves as a common architectural and design reference for all stakeholders involved in the development, testing, review, and continued evolution of the system.

1.4 Definitions and Abbreviations

The following terms and abbreviations are used throughout this Software Architecture and Design Specification (SAD):

Term / Abbreviation	Definition
ML	Machine Learning
UI	User Interface
API	Application Programming Interface
CSV	Comma-Separated Values, a tabular file format supported for dataset upload
SRS	Software Requirements Specification
STP	Software Test Plan
RTM	Requirements Traceability Matrix
SAD	Software Architecture and Design Specification
TLS	Transport Layer Security, used to secure communication between the client and server
Dataset	A collection of structured data used as input for machine-learning processing and evaluation
Feature	An input variable or attribute used by a machine-learning model
Target	The output variable that a machine-learning model is intended to predict
Classification	A supervised machine-learning task in which the model predicts a category or class
Regression	A supervised machine-learning task in which the model predicts a continuous numerical value
Preprocessing	The process of preparing and transforming dataset features before model training, including missing-value handling, encoding, scaling, and data splitting
Training	The process of fitting a selected machine-learning model using the prepared training dataset
Testing	The process of evaluating a trained model using previously unseen held-out test data
Evaluation Metric	A quantitative measure used to assess the performance of a trained machine-learning model
Model Run	A single execution of the model training and/or evaluation workflow
Artifact	A file or generated output produced during model training or evaluation, such as a trained model or report
Audit Event	A recorded security or system activity, such as a login, access attempt, or administrative action
CI/CD	Continuous Integration / Continuous Delivery or Deployment, referring to automated software integration and delivery workflows
Celery	A background task-processing system that can be used to execute long-running operations asynchronously
Redis	An in-memory data store that can be used as a message broker for background task processing

These definitions establish the terminology used consistently throughout the architecture, design, interface, security, and implementation-related sections of this document.

2. Document Overview

2.1 How to Use This Document

This Software Architecture and Design Specification (SAD) is organized to provide a structured description of the architecture and detailed design of the **Web Based ML Model Evaluator**. The document should be read as a continuation of the Software Requirements Specification (SRS) and Software Test Plan (STP), with the architecture and design decisions presented here derived from the requirements and testing needs identified in those documents.

The document is organized into the following major sections:

- **Section 1 – Introduction:** Defines the purpose, scope, intended audience, and terminology used throughout the SAD.
- **Section 2 – Document Overview:** Explains how the SAD is organized and identifies the related project documents that should be considered along with this specification.
- **Section 3 – Architecture:** Describes the architectural goals and constraints, stakeholders and their concerns, major system components, component responsibilities, selected architecture pattern, technology stack, data stores, risks and mitigations, requirements traceability, and security architecture.
- **Section 4 – Design:** Presents the detailed design of the system, including the overall design approach, UML sequence diagrams, API interfaces, error handling, logging and monitoring, user-experience considerations, and open issues or future design considerations.
- **Section 5 – Appendices:** Provides supporting information such as the project glossary, references, and tools used in the architecture and design process.

For understanding the system, the recommended reading order is to first review the **Introduction** to understand the purpose and scope of the document, followed by the **Architecture** section to understand the overall structure and organization of the system. The **Design** section can then be used to understand the interactions between components, interfaces, workflows, error handling, and user-experience considerations. The appendices provide supporting terminology and reference information.

The SAD should be used together with the **Software Requirements Specification (SRS)**, **Software Test Plan (STP)**, and **Requirements Traceability Matrix (RTM)**. The SRS defines what the system is required to do, the STP defines how the system will be verified and tested, and this SAD defines how the requirements are organized into an architectural and design solution. This relationship helps maintain consistency between requirements, architecture, implementation, and testing throughout the project lifecycle.

2.2 Related Documents

The following documents and project artifacts are directly related to the **Web Based ML Model Evaluator** and are used as supporting references for this Software Architecture and Design Specification (SAD).

Related Document / Artifact	Phase / Version	Purpose and Relationship to SAD
Software Requirements Specification (SRS)	Phase 1 – Version 1.0, dated 06-09-2026	Defines the functional and non-functional requirements, system scope, interfaces, features, acceptance criteria, UML use-case diagrams, and Requirements Traceability Matrix (RTM). The SAD translates these requirements into the proposed architecture, components, interfaces, and design decisions. Team-13_Title-(Web_Based_ML_Mod...
Software Test Plan (STP)	Phase 2 – Version 1.1, dated 16-09-2026	Defines the testing scope, test strategy, test environment, test schedule, responsibilities, risks, test deliverables, and test traceability. The SAD provides the architectural and workflow basis required to support the testing activities defined in the STP. Team-13_Title-(Web_Based_ML_Mod...
Requirements Traceability Matrix (RTM)	Phase 1 – Included in SRS Version 1.0	Maps functional, non-functional, and security requirements to design specifications, system modules, and test cases. It provides the traceability link between the requirements, the architecture/design defined in the SAD, and the testing activities defined in the STP. Team-13_Title-(Web_Based_ML_Mod...
UML Use-Case Diagrams	Phase 1 – Included in SRS Version 1.0	Represents the major user, project, data-management, model-evaluation, and results workflows. These diagrams provide the functional context for defining the architectural components and detailed interactions in the SAD. Team-13_Title-(Web_Based_ML_Mod...
Faculty Software Architecture and Design Specification (SAD) Template	Phase 3 – Reference Template	Provides the prescribed structure and expected contents for the current SAD, including architecture goals and constraints, stakeholders, component UML diagram, component descriptions, architecture rationale, technology stack, risks, traceability, security architecture, sequence diagrams, API design, error handling, UX design, and appendices.
Software Architecture and Design Specification (SAD)	Phase 3 – Version 1.2, current document	Defines the architectural and detailed design solution for the system based on the requirements and testing documents from the previous phases. It establishes the architecture baseline, components, interfaces, data flow, security controls, design decisions, and supporting UML and design specifications.

Document relationship: Phase 1 establishes **what the system must do**, Phase 2 establishes **how the system will be tested**, and Phase 3 establishes **how the system is architected and designed** to satisfy those requirements and support the defined testing activities. The RTM maintains traceability across these phases.

3. Architecture

3.1 Goals & Constraints

The architecture of the **Web Based ML Model Evaluator** is designed to provide a secure, reliable, modular, and maintainable platform for the complete supervised machine-learning evaluation workflow. The architectural goals and constraints are derived from the functional requirements, non-functional requirements, and project scope defined in the SRS.

Architectural Goals

The proposed architecture aims to achieve the following goals:

1. **Secure Access:** Provide secure user authentication, authorization, and controlled access to projects, datasets, model information, and evaluation results.
2. **Reliable Processing:** Ensure that dataset processing, preprocessing, model training, testing, and evaluation operations execute consistently and handle failures appropriately.
3. **Modularity:** Organize the system into clearly defined components with separate responsibilities so that individual modules can be developed, tested, maintained, and modified independently.
4. **Correct Model Evaluation:** Ensure that trained models are evaluated using previously unseen held-out test data and that appropriate classification or regression metrics are calculated correctly.
5. **Understandable Results:** Present evaluation results through clear tables, visualizations, comparisons, and reports so that users can understand model performance.
6. **Maintainability:** Use well-defined component boundaries and interfaces so that the system can be maintained and extended without unnecessary impact on other components.
7. **Performance:** The architecture should support the performance requirements defined in the SRS. Commonly used application pages should complete within **3 seconds for at least 90% of requests**, and dataset previews should be generated within **3 seconds for supported datasets up to 10 MB**, excluding upload time.
8. **Reliability and Data Integrity:** The system should maintain at least **99% scheduled availability**, excluding planned maintenance, while preserving the consistency of project, model-run, dataset, and evaluation-result information after completed operations.

Architectural Constraints

The architecture is subject to the following constraints:

1. **Dataset Constraint:** The initial system is designed primarily for **tabular datasets**, with CSV as the supported dataset format.
2. **Machine-Learning Scope:** The system is limited to **supervised machine-learning tasks**, specifically classification and regression.

3. **Supported Models and Libraries:** Only machine-learning models and preprocessing techniques supported by the selected machine-learning libraries are included in the initial implementation.
4. **Resource Constraint:** The maximum practical dataset size and computational workload depend on the available server CPU, memory, storage, and other system resources.
5. **Scope Constraint:** Deep-learning models, image/video processing, real-time data evaluation, and production deployment of trained models are outside the initial scope of the system.
6. **Input Validation Constraint:** Uploaded datasets and user inputs must be validated before processing to prevent invalid or unsupported data from entering the processing and machine-learning workflow.
7. **Web-Based Access:** The system is designed to operate through modern web browsers, with communication between the user interface and backend handled through defined application interfaces.

These goals and constraints establish the architectural boundaries for the subsequent component structure, architecture pattern, technology decisions, interfaces, security architecture, and detailed system design presented in this SAD.

unctions.

3.2 Stakeholders & Concerns

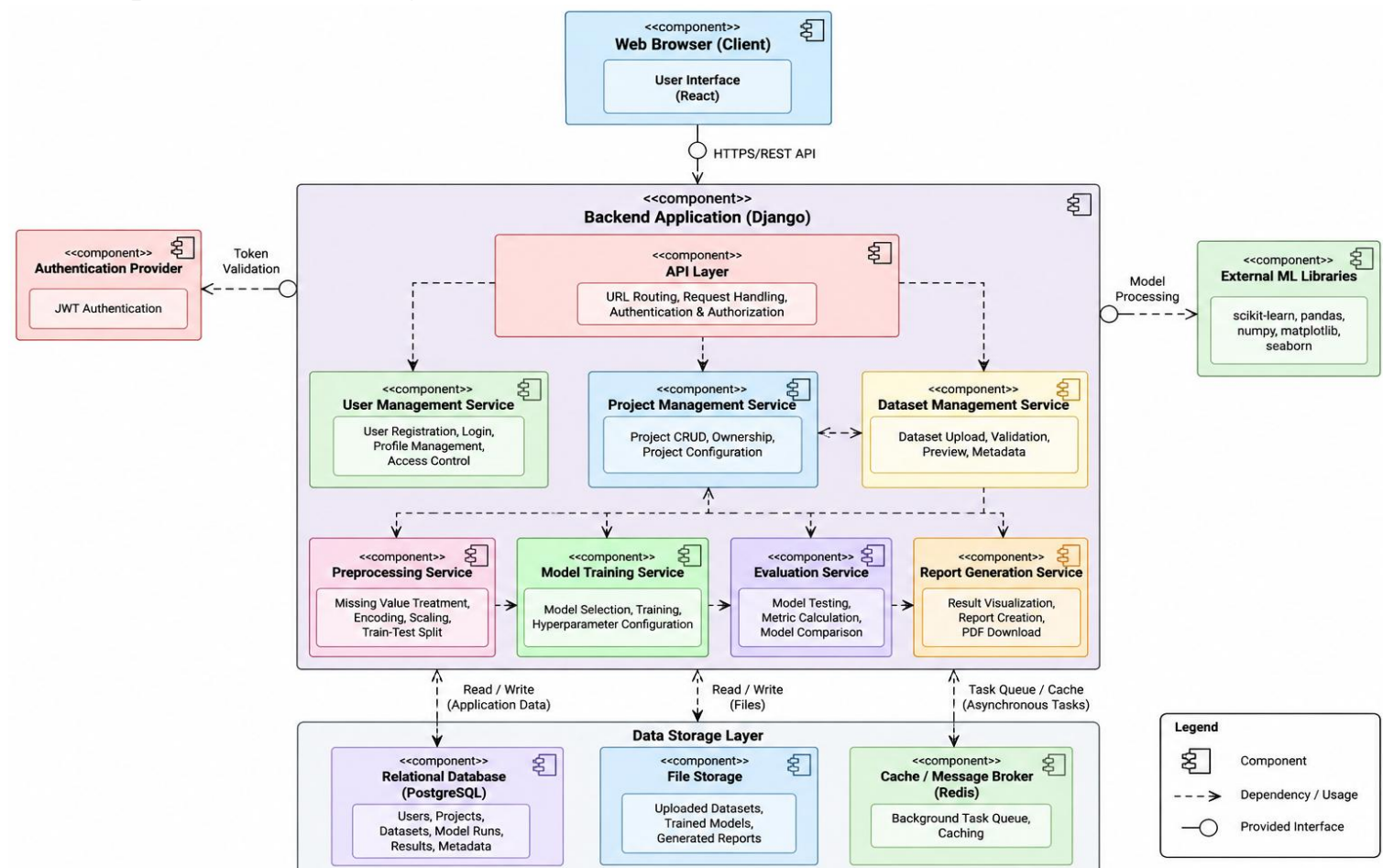
The architecture of the **Web Based ML Model Evaluator** must address the needs of different stakeholders involved in using, developing, testing, evaluating, and maintaining the system. The major stakeholders and their primary architectural concerns are described below.

Stakeholder	Primary Concerns
Registered Users / ML Practitioners	Ease of use, clear navigation, simple dataset upload and preprocessing, correct model selection, reliable training and evaluation, understandable metrics and visualizations, comparison of results, and generation of reports.
Project Guide / Evaluator	Correct implementation of the defined requirements, consistency of the complete system workflow, accuracy of model-evaluation results, requirement traceability, appropriate architectural and design decisions, and availability of suitable test evidence.
Administrator	Secure access control, management of users and project records, protection of stored datasets and evaluation information, system monitoring, and maintenance of operational records.
Developers / ML Engineers	Clear component responsibilities, modular architecture, well-defined interfaces, manageable dependencies, maintainable implementation, correct integration of data-processing and machine-learning modules, and ease of future enhancement.

QA Engineers / Test Coordinators	Testable component boundaries, clearly defined interfaces and workflows, reliable error handling, traceability between requirements and system components, and support for functional, integration, system, performance, security, usability, and data-integrity testing.
Academic Users / Future Maintainers	Understandable system structure, maintainable components, clear documentation, ease of modification, and the ability to extend the system with additional machine-learning capabilities in future versions.

The architecture therefore needs to balance **usability, security, correctness, performance, maintainability, testability, and extensibility**. These concerns are consistent with the quality attributes and stakeholder expectations established in the SRS and the testing activities defined in the STP. The SRS specifically identifies users, ML practitioners, project evaluators, administrators, developers, and testers as important concerns for the system architecture. Team-13_Web-Based-ML-Model-Eval...

3.3 Component (UML) Diagram



3.4 Component Descriptions

The architecture of the **Web Based ML Model Evaluator** is organized into modular components, with each component responsible for a specific part of the system workflow. The components communicate through defined interfaces and exchange the data required for dataset processing, model training, evaluation, and reporting. The following descriptions correspond to the components shown in the UML component diagram.

Component	Description	Main Responsibilities
Web Browser / Client	Provides the user-facing environment through which users interact with the Web Based ML Model Evaluator.	User interaction, navigation, project management, dataset upload, preprocessing configuration, model selection, training-status display, result visualization, and report access.
User Interface	Provides the presentation layer of the application and displays forms, tables, charts, status information, and evaluation results to the user.	Registration/login screens, dashboard, dataset preview, preprocessing controls, model configuration, results display, visualizations, and report download.
Backend Application	Acts as the central application layer responsible for coordinating requests from the client and invoking the appropriate system services.	Request processing, workflow coordination, business logic, communication with storage, security enforcement, and coordination of machine-learning operations.
API Layer	Provides the application interfaces through which the client communicates with the backend services.	URL routing, request handling, authentication and authorization checks, input validation, structured responses, and communication between the user interface and backend services.
Authentication Provider	Handles authentication-related operations and supports secure access to the application and protected resources.	User authentication, token/session validation, access control, and prevention of unauthorized access to projects, datasets, and evaluation results.
User Management Service	Manages user-related functionality within the application.	User registration, login-related operations, profile management, and user access control.
Project Management Service	Manages machine-learning evaluation projects created by authenticated users.	Project creation, viewing, renaming, deletion/management, project ownership, and project configuration.
Dataset Management Service	Handles the complete lifecycle of uploaded datasets before they are passed to the machine-learning workflow.	CSV upload, file validation, dataset preview, dataset metadata management, and association of datasets with projects.
Preprocessing Service	Prepares uploaded datasets for machine-learning model training and testing.	Missing-value treatment, categorical-data encoding, feature scaling, target selection support, and train-test splitting.
Model Training Service	Handles model selection, configuration, and training using the prepared dataset.	Selection of compatible classification or regression models, parameter configuration, model training, training-status handling, and trained-model management.

Evaluation Service	Tests trained models using held-out test data and calculates the appropriate performance measures.	Model testing, classification/regression metric calculation, evaluation-result generation, and comparison of model performance.
Report Generation Service	Converts evaluation results into understandable visual and downloadable outputs.	Result visualization, report creation, preparation of evaluation summaries, and report download.
External ML Libraries	Provides the data-processing and machine-learning functionality required by the backend. The SRS identifies Pandas, NumPy, and scikit-learn for these purposes.	Dataset loading and analysis, preprocessing operations, model training, model testing, and calculation of evaluation metrics.
Relational Database	Stores structured application information required for persistent system operation.	User information, projects, dataset metadata, model-run information, evaluation results, and related application records.
File Storage	Stores files and generated artifacts that are not appropriate for direct storage as relational records.	Uploaded datasets, trained-model artifacts where required, and generated evaluation reports.
Cache / Message Broker	Supports background processing and temporary task-management requirements when longer-running operations are separated from normal web requests.	Background task queue, task coordination, and caching. The SRS specifies Celery with Redis as an option if longer training or evaluation operations require asynchronous processing.

Component Interaction

The major interaction flow between these components is: **Web Browser / User Interface → API Layer → Backend Services → ML Libraries / Data Storage**

For the machine-learning workflow, the primary processing path is: **Dataset Management → Preprocessing → Model Training → Evaluation → Report Generation**

The backend services use the **Relational Database** for structured application records and **File Storage** for uploaded datasets and generated artifacts. Longer-running training or evaluation operations may use the **Cache / Message Broker** for asynchronous task processing. This modular separation supports the maintainability, testability, and independent management of the major system functions required by the SRS.

3.5 Chosen Architecture Pattern and Rationale

The **Web Based ML Model Evaluator** adopts a **Layered Modular Architecture**. This architecture organizes the system into logical layers, where each layer has a defined responsibility and communicates with the appropriate adjacent layer through well-defined interfaces.

The selected architecture is suitable for the project because the system contains distinct responsibilities such as user interaction, application/API processing, machine-learning operations, data persistence, and file management. The modular organization also supports independent development, testing, maintenance, and future enhancement. The project draft explicitly identifies a layered modular architecture as the selected pattern.

Architectural Layers

Layer	Responsibility
Presentation Layer	Provides the web-based user interface through which users interact with the application, including project management, dataset upload, preprocessing configuration, model selection, and result visualization.
API / Application Layer	Handles requests from the presentation layer, authentication and authorization checks, request validation, routing, and coordination of application operations.
Domain Services Layer	Contains the major application services such as user management, project management, dataset management, preprocessing, model selection, training, evaluation, and reporting.
Processing Layer	Performs dataset processing and machine-learning operations using Pandas, NumPy, and scikit-learn.
Persistence Layer	Manages persistent application data, dataset metadata, model-run information, evaluation results, and files through the selected database and file-storage mechanisms.

Rationale for Selection

The layered modular architecture was selected for the following reasons:

1. **Clear Separation of Responsibilities:** Each layer performs a specific responsibility, reducing unnecessary coupling between the user interface, application logic, machine-learning processing, and data storage.
2. **Maintainability:** Individual modules can be modified or maintained without requiring major changes to unrelated parts of the system.
3. **Testability:** Clearly separated components make it easier to perform unit, integration, and system-level testing. This supports the testing strategy defined in the STP.
4. **Scalability of Processing:** Machine-learning operations such as preprocessing, training, and evaluation can be separated from the presentation and application layers. Longer-running operations can also use background processing where required.
5. **Security:** Authentication, authorization, input validation, and access-control checks can be enforced at appropriate application boundaries before protected resources are accessed.
6. **Consistency with the ML Workflow:** The architecture naturally supports the project's sequential workflow: **Dataset Upload** → **Validation/Preview** → **Preprocessing** → **Model Selection** → **Training** → **Testing** → **Evaluation** → **Comparison** → **Visualization/Reporting**

7. **Future Extensibility:** The modular structure allows additional models, preprocessing techniques, evaluation metrics, visualization capabilities, or other services to be incorporated without redesigning the entire application.
8. **Alignment with Project Requirements:** The SRS already defines separate web-interface, backend, data-processing, model-selection/training, evaluation, visualization, and database-storage responsibilities, making a layered organization appropriate for translating those requirements into the system architecture. Team-13_Title-(Web_Based_ML_Mod...

Overall, the **Layered Modular Architecture** provides a balance between simplicity, maintainability, testability, and future extensibility while remaining appropriate for the current scope of the Web Based ML Model Evaluator.

3.6 Technology Stack & Data Stores

The technology stack for the **Web Based ML Model Evaluator** is selected to support the complete workflow of dataset management, preprocessing, supervised machine-learning model training, evaluation, visualization, and report generation. The technology choices are aligned with the system requirements and the layered modular architecture defined in the previous sections.

Technology Stack

Layer / Area	Technology	Purpose
Programming Language	Python	Core backend programming language and implementation of data-processing and machine-learning functionality.
Backend Framework	Django	Provides the web application backend, request handling, application logic, authentication support, and integration between system components.
Frontend	HTML, CSS, JavaScript, Bootstrap	Provides the browser-based user interface, forms, navigation, dataset interaction, configuration screens, and result presentation.
Data Processing	Pandas, NumPy	Used for dataset loading, analysis, transformation, and preprocessing operations.
Machine Learning	scikit-learn	Provides supported classification and regression models, model training, testing, and evaluation functionality.
Data Visualization	Chart.js / Plotly.js	Used to present model-performance comparisons and evaluation results through graphical visualizations.
API / Communication	HTTP/HTTPS, JSON	Used for communication between the web interface and backend application and for structured data exchange where applicable.
Background Processing	Celery with Redis (<i>if required</i>)	Supports longer-running model-training or evaluation operations without unnecessarily blocking the web interface.
Version Control	Git and GitHub	Used for source-code version control, collaboration, and project management.

The SRS identifies Python with Django for the backend, Pandas and NumPy for data processing, scikit-learn for machine learning, HTML/CSS/JavaScript with Bootstrap or another suitable web frontend, and Chart.js or Plotly.js for visualization.

Data Stores

The system uses separate storage mechanisms according to the type of information being stored.

Data Store	Stored Information	Purpose
Relational Database	User information, projects, dataset metadata, preprocessing configurations, model-run information, evaluation metrics, preferred-model information, and evaluation history	Provides structured and persistent storage for application records and maintains relationships between system entities.
File Storage	Uploaded CSV datasets, trained-model artifacts where required, and generated evaluation reports	Stores larger files and generated artifacts separately from structured database records.
Redis (<i>if background processing is required</i>)	Temporary task and queue information	Supports asynchronous processing through Celery for longer-running training or evaluation operations.

The SRS specifies **SQLite or PostgreSQL** as the database alternatives for storing user information, projects, dataset metadata, model runs, and evaluation results. It also specifies file storage for uploaded datasets and generated model artifacts where required. Celery with Redis may be introduced if longer-running training and evaluation tasks need background processing.

Technology Selection Rationale

The selected technologies support the architectural goals in the following ways:

- **Python, Pandas, NumPy, and scikit-learn** provide a consistent environment for the data-processing and machine-learning workflow.
- **Django** provides a structured backend suitable for implementing authentication, project management, APIs, business logic, and database interaction.
- **HTML, CSS, JavaScript, and Bootstrap** provide a straightforward web interface suitable for the project's academic and functional scope.
- **Chart.js or Plotly.js** supports understandable presentation of model-comparison and evaluation results.
- **Relational database storage** provides structured persistence and relationships between users, projects, datasets, model runs, and evaluation results.
- **Separate file storage** avoids placing large dataset and model files directly inside relational records.
- **Celery with Redis**, where required, allows computationally longer training operations to execute without unnecessarily blocking the user's web session.

3.7 Risks & Mitigations

The following risks are considered in the architecture and design of the **Web Based ML Model Evaluator**. The mitigation strategies are intended to reduce the impact of data-processing failures, long-running machine-learning operations, security issues, integration problems, and uncontrolled expansion of the project scope.

Risk	Potential Impact	Mitigation
Invalid or oversized dataset	Invalid, corrupted, unsupported, or excessively large datasets may cause processing failures or excessive resource consumption.	Validate file type, size, structure, and parsability before storing or processing the dataset. Reject invalid inputs with clear error messages.
Long-running model training	Large datasets or computationally expensive models may take considerable time and may block normal user interaction.	Maintain training-status information and use background processing such as Celery with Redis where required for longer-running operations.
Unauthorized access to user data	Users may gain access to projects, datasets, model information, or evaluation results belonging to another user.	Enforce authentication, authorization, and project-ownership checks before allowing access to protected resources.
Frontend-backend interface mismatch	Inconsistent request formats, response structures, identifiers, or status handling may cause integration failures.	Maintain clearly defined API contracts, consistent identifiers, HTTP status codes, request/response formats, and error structures between the frontend and backend.
Inconsistent stored results	Failures during processing or database operations may result in inconsistent project, model-run, or evaluation-result information.	Use appropriate database relationships, transactions, integrity constraints, and validation checks to maintain data consistency.
Incorrect model evaluation	Incorrect preprocessing, data splitting, or metric calculation could produce misleading model-performance results.	Use held-out test data for final evaluation, enforce task-compatible models and metrics, and verify the evaluation workflow through functional and integration testing.
Insufficient system resources	CPU, memory, storage, or other resource limitations may restrict the size of datasets that can be processed successfully.	Define practical dataset-size limits, validate resource-intensive operations, and consider background processing for longer-running workloads.
Security and input-validation failures	Malicious or malformed inputs could affect application security or expose sensitive information.	Apply input validation, file validation, authentication, authorization, HTTPS/TLS communication, secure password handling, and security logging as defined in the SRS.
Scope expansion	Adding unsupported features such as deep learning, real-time evaluation, or image/video processing may increase complexity and affect project completion.	Maintain the defined project scope and RTM, and explicitly defer features outside the initial scope to future enhancements.

Failure during report or artifact generation	Generated reports or model artifacts may be unavailable or incomplete after an evaluation operation.	Maintain appropriate processing-status and error handling, store generated artifacts separately where required, and verify successful generation before making them available to users.
---	--	---

The above risks directly support the architectural goals of **security, reliability, maintainability, data integrity, performance, and controlled scope**. The risk treatment also supports the testing activities defined in the STP, particularly functional, integration, performance, security, usability, reliability, and data-integrity testing.

3.8 Traceability to Requirements

The architecture and design of the **Web Based ML Model Evaluator** are derived from the requirements defined in the SRS. The Requirements Traceability Matrix (RTM) establishes the relationship between each requirement, its design specification, corresponding system module, and test case. This section summarizes how the requirements are addressed by the architectural components defined in this SAD. The SRS RTM contains **26 functional requirements, 7 non-functional requirements, 2 security objectives, and 6 security requirements**.

Functional Requirements Traceability

Requirement IDs	Requirement Area	Architectural Component(s)	Related Test Cases
ML-F-001 – ML-F-004	User registration, authentication, project management, and logout	User Management Service, Authentication Provider, Project Management Service, API Layer	TC-Auth-01, TC-Auth-02, TC-Auth-03
ML-F-010 – ML-F-013	Dataset upload, validation, preview, target and task selection	Dataset Management Service, API Layer, File Storage	TC-Data-01 to TC-Data-04
ML-F-014	Preprocessing configuration and train-test split	Preprocessing Service, External ML Libraries	TC-Data-05
ML-F-020 – ML-F-023	Classification/regression model catalogue, model selection, and parameter configuration	Model Training Service, External ML Libraries	TC-Model-01 to TC-Model-04
ML-F-030 – ML-F-033	Model training, background execution, training status, and model-run storage	Model Training Service, Cache/Message Broker, Relational Database, File Storage	TC-Train-01 to TC-Train-04
ML-F-040 – ML-F-043	Model testing, classification/regression metrics, and evaluation-result storage	Evaluation Service, External ML Libraries, Relational Database	TC-Eval-01 to TC-Eval-04
ML-F-050 – ML-F-051	Model-result comparison and visualization	Evaluation Service, Report Generation Service, Web UI	TC-Result-01, TC-Result-02
ML-F-052	Preferred-model selection	Report Generation Service / Results Management, Relational Database	TC-Result-03

ML-F-053	Evaluation report generation and download	Report Generation Service, File Storage	TC-Report-01
ML-F-054	Retrieval of previous evaluation results	Evaluation Service, Relational Database, Web UI	TC-Result-04

The requirement groupings and associated test cases are based on the functional RTM and STP. For example, the RTM maps ML-F-014 to the preprocessing module, ML-F-020–023 to model-selection/configuration modules, ML-F-030–033 to training-related modules, and ML-F-040–043 to testing/evaluation modules.

Non-Functional Requirements Traceability

Requirement ID	Requirement Area	Architectural Support	Related Test Case
ML-NF-001	Application pages should load within 3 seconds for 90% of requests	Web UI, API Layer, Backend Application	TC-Perf-01
ML-NF-002	Dataset preview within 3 seconds for supported datasets up to 10 MB, excluding upload time	Dataset Management Service, Data Processing Layer	TC-Perf-02
ML-NF-003	At least 99% scheduled availability excluding planned maintenance	Backend Application, Database, System Infrastructure	TC-Avail-01
ML-NF-004	Clear error messages without exposing internal system details	API Layer, Backend Services, Web UI	TC-UX-01
ML-NF-005	Keyboard navigation, descriptive labels, and sufficient colour contrast	Web UI / Presentation Layer	TC-UX-02
ML-NF-006	Consistency of project, model-run, and evaluation-result information	Relational Database, Data Management Components	TC-Data-06
ML-NF-007	Modular and maintainable architecture	Layered Architecture and independent system components	TC-Maint-01

These requirements correspond to the measurable performance, reliability, usability, accessibility, data-integrity, and maintainability requirements defined in the SRS.

Security Requirements Traceability

Security ID	Security Requirement / Objective	Architectural Support	Related Test Case
SO-001	Protect user accounts, datasets, model artifacts, and results from unauthorized access	Authentication Provider, Authorization, API Layer, protected storage	TC-Sec-03
SO-002	Protect datasets, configurations, evaluation results, and project information from unauthorized modification	Authorization, Data Integrity Controls, Relational Database	TC-Sec-04
PRJ-SR-001	Enforce HTTPS with TLS 1.2 or higher	Communication / Security Layer	TC-Sec-01
PRJ-SR-002	Securely hash user passwords and prohibit plaintext storage	Authentication Provider / User Management Service	TC-Sec-02
PRJ-SR-003	Restrict access to user-owned resources	Authorization / Project Management Service	TC-Sec-03

PRJ-SR-004	Validate uploaded dataset files	Dataset Management and Dataset Validation Services	TC-Sec-04
PRJ-SR-005	Validate user inputs and model parameters	API Layer, Input Validation, Model Configuration	TC-Sec-05
PRJ-SR-006	Maintain security audit logs	Audit / Logging Component	TC-Sec-06

The security objectives and security requirements are defined in the SRS RTM, including confidentiality, integrity, HTTPS/TLS protection, password security, resource-level access control, dataset validation, input validation, and security logging.

Traceability Summary

The traceability established above ensures that the architecture is directly connected to the requirements defined in the SRS and the verification activities defined in the STP. Each major functional workflow is represented by one or more architectural components, while the non-functional and security requirements are addressed through cross-cutting architectural concerns such as performance, modularity, validation, access control, data integrity, and secure communication.

The RTM therefore provides the link: **SRS Requirement → Architectural Component / Design → Implementation → Test Case**

This traceability will be maintained throughout implementation and testing so that changes to requirements or architecture can be evaluated for their effect on corresponding components and test activities. The SRS RTM confirms that design specifications, system modules, and test-case references are provided for the defined requirement categories.

3.9 Security Architecture

Security is an integral part of the architecture of the **Web Based ML Model Evaluator** because the system handles user accounts, uploaded datasets, model configurations and artifacts, evaluation results, and generated reports. The security architecture is designed around the two primary security objectives defined in the SRS: **confidentiality** and **integrity**.

3.9.1 Security Objectives

The architecture addresses the following security objectives:

- **SO-001 – Confidentiality:** Protect user accounts, uploaded datasets, model artifacts, and evaluation results from unauthorized access.
- **SO-002 – Integrity:** Prevent unauthorized modification or tampering with datasets, model configurations, evaluation results, and stored project information.

3.9.2 Security Controls

Security Control	Architectural Implementation	Requirement
Secure Communication	Client-server communication uses HTTPS with TLS 1.2 or higher to protect data transmitted between the browser and application backend.	PRJ-SR-001
Password Protection	User passwords are stored using a suitable one-way password-hashing mechanism and are never stored in plaintext.	PRJ-SR-002
Authentication	Users must authenticate before accessing protected application functionality and user-specific resources.	PRJ-SR-002, PRJ-SR-003
Authorization and Ownership Checks	Access to projects, datasets, model runs, results, and reports is restricted to authorized users and the resources they are permitted to access.	PRJ-SR-003
Dataset/File Validation	Uploaded files are validated for supported format, validity, corruption, and permitted size before processing.	PRJ-SR-004
Input and Parameter Validation	User inputs and model parameters are validated before being passed to application or machine-learning processing components.	PRJ-SR-005
Security Logging	Important security events such as login attempts, access violations, and administrative actions are recorded with relevant event information and timestamps.	PRJ-SR-006
Sensitive Information Protection	Passwords, authentication tokens, and other sensitive credentials are not included in security logs or error messages.	PRJ-SR-006

These controls directly correspond to the security requirements defined in the SRS.

3.9.3 Security Across the System Architecture

Security controls are applied at multiple architectural boundaries rather than being restricted to a single component.

Web Browser → API Layer

- Communication is protected using HTTPS/TLS.
- Requests to protected resources require authentication.
- User inputs are validated before processing.

API Layer → Application Services

- Authorization and project-ownership checks are applied before accessing user-specific resources.
- Invalid requests and model parameters are rejected.
- Application errors are returned without exposing sensitive internal information.

Dataset Management → Processing Layer

- Uploaded files are validated before they enter the data-processing pipeline.

- Unsupported, corrupted, invalid, or oversized files are rejected.
- Dataset-related operations are associated with the appropriate authorized project.

Application Services → Data Stores

- Access to structured application information is controlled through the application's authorization mechanisms.
- User, project, dataset, model-run, and evaluation-result information must remain associated with the appropriate authorized user/project.
- Stored information must maintain integrity after completed operations.

Application → Logging

- Security-relevant events are recorded for auditing and investigation.
- Sensitive credentials and confidential information are excluded from logs.

3.9.4 Threat Considerations

The architecture also considers the following major threat categories:

Threat	Mitigation
Spoofing	User authentication and secure session/access mechanisms.
Tampering	Authorization, ownership checks, input validation, and data-integrity controls.
Information Disclosure	HTTPS/TLS, protected storage, access control, and avoidance of sensitive information in errors/logs.
Denial of Service	Dataset/file-size validation and controlled processing of resource-intensive operations.
Privilege Escalation	Role and authorization checks before accessing protected functionality and resources.

These threat considerations and corresponding mitigations are consistent with the security architecture described in the project's working SAD material.

3.9.5 Security Validation

The security architecture is verified through the security test cases defined in the SRS and STP. Testing covers authentication, authorization, password protection, HTTPS communication, dataset/file validation, input validation, data protection, and security logging.

The overall security flow can therefore be summarized as: **Authenticate User → Authorize Resource Access → Validate Input/File → Process Request → Protect Stored Data → Record Security Event**

This layered approach ensures that security is considered throughout the complete application workflow rather than only at the user-login stage.

4. Design

4.1 Design Overview

The detailed design of the **Web Based ML Model Evaluator** translates the architectural structure defined in Section 3 into concrete system interactions, interfaces, data flow, and processing behaviour. The design follows the **Layered Modular Architecture** selected for the system and is based on the functional workflow, interfaces, and integration flows defined in the SRS and STP.

The design is organized around the complete machine-learning evaluation workflow: **Authenticate → Create Project → Upload Dataset → Validate & Preview → Preprocess & Split → Select ML Task & Model → Train → Test → Evaluate → Compare & Visualize → Generate Report**

This workflow corresponds to the end-to-end system-testing flow defined in the STP.

4.1.1 Design Approach

The system is designed using clearly separated modules so that each major responsibility can be implemented, tested, and maintained independently.

The primary design areas are:

- **User and Access Management:** Handles registration, authentication, logout, session management, and authorization.
- **Project Management:** Maintains user-owned machine-learning evaluation projects and their associated information.
- **Dataset Management:** Handles CSV upload, validation, storage, preview, and basic dataset analysis.
- **Data Preprocessing:** Handles target-column selection, task selection, missing-value treatment, categorical encoding, feature scaling, and train-test splitting.
- **Model Selection and Configuration:** Provides compatible classification and regression models and allows relevant model parameters to be configured.
- **Model Training:** Trains the selected models using the prepared training dataset and maintains training status.
- **Model Testing and Evaluation:** Tests trained models using held-out test data and calculates appropriate classification or regression metrics.
- **Results Management:** Stores evaluation results and supports comparison, visualization, preferred-model selection, and evaluation-history access.
- **Report Generation:** Produces downloadable evaluation reports containing relevant dataset, model, parameter, metric, and result information.

- **Error Handling and Status Management:** Provides appropriate feedback when validation, preprocessing, training, or evaluation operations fail.

These design areas correspond to the system functions and integration points defined in the SRS and STP.

4.1.2 Component Interaction and Data Flow

The major interaction between the components is designed as follows: **User Interface → API/Application Layer → Domain Services → Processing/Data Layer**

The primary processing flow is: **Dataset Management → Preprocessing → Model Selection → Training → Testing → Evaluation → Results → Reporting**

The major integration relationships are:

- **Authentication → Project Management:** Only authenticated users can manage their evaluation projects.
- **Project Management → Dataset Management:** Uploaded datasets are associated with the selected project.
- **Dataset Upload → Validation → Preview:** Uploaded files are validated before their contents are processed or displayed.
- **Preprocessing → Model Selection:** The selected task and prepared dataset determine compatible models.
- **Preprocessing → Model Training:** Prepared training data is supplied to the selected model.
- **Model Training → Model Testing:** A successfully trained model is evaluated using the held-out test dataset.
- **Model Testing → Evaluation:** Test predictions are used to calculate appropriate performance metrics.
- **Evaluation → Comparison and Visualization:** Evaluation results are presented through tables and graphical representations.
- **Evaluation Results → Report Generation:** Stored results are used to generate downloadable evaluation reports.
- **Project/Data → Database and File Storage:** Structured records are persisted in the database while uploaded datasets and generated artifacts are stored using file storage where required.

4.1.3 Interface and Communication Design

The web interface communicates with the backend through defined application interfaces. The SRS specifies HTTP/HTTPS-based communication, JSON for structured API data where applicable, and multipart upload for dataset files. HTTPS with TLS 1.2 or higher is used to protect client-server communication.

Long-running model-training or evaluation operations should provide processing-status information to the user. Where required, background processing using **Celery with Redis** may be introduced so that computationally intensive operations do not unnecessarily block the web interface.

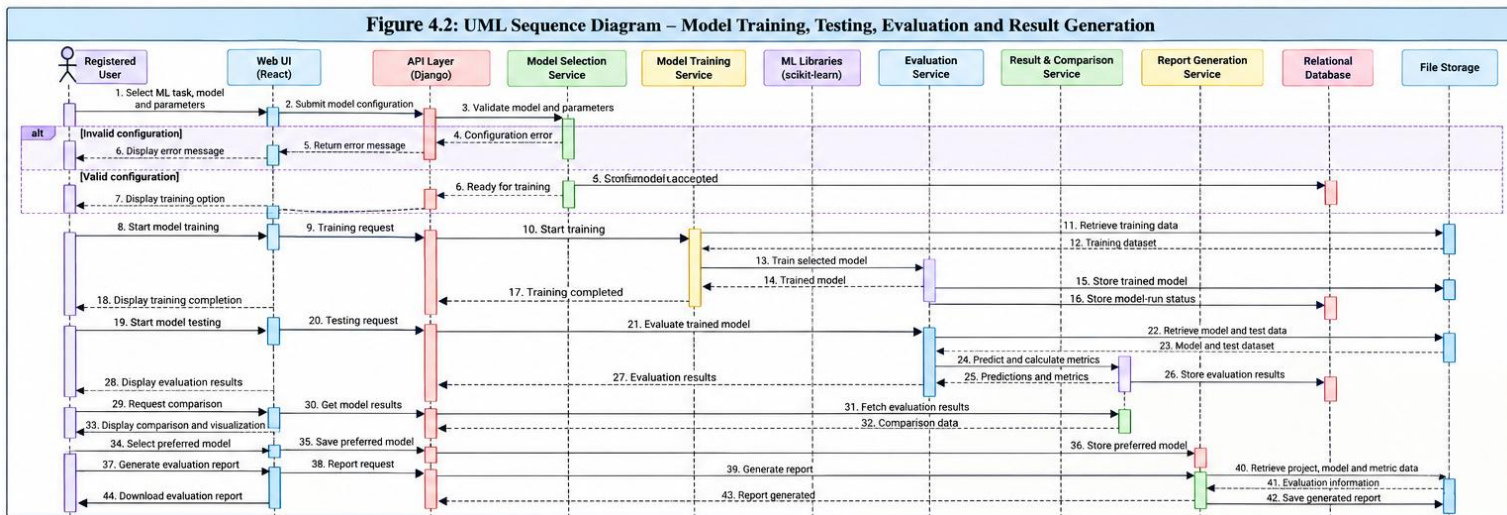
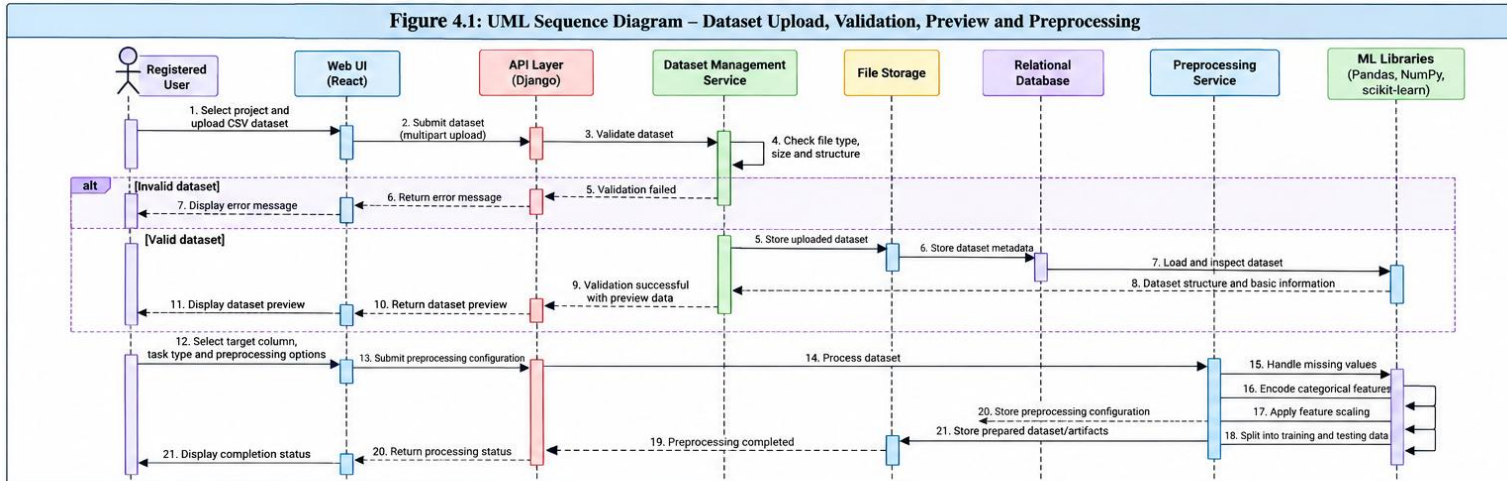
4.1.4 Design Principles

The detailed design follows these principles:

1. **Separation of concerns** – UI, application logic, machine-learning processing, and persistence responsibilities are kept distinct.
2. **Modularity** – Major system functions are implemented as independent components with defined responsibilities.
3. **Input validation** – Dataset files, user inputs, and model parameters are validated before processing.
4. **Secure access** – Authentication and authorization are applied before protected resources are accessed.
5. **Data integrity** – Project, dataset, model-run, and evaluation information is maintained consistently.
6. **Testability** – Component boundaries and interfaces support functional, integration, and system testing.
7. **Extensibility** – The design allows additional supported models, preprocessing methods, and evaluation capabilities to be added without restructuring the entire system.
8. **User feedback** – Processing status, validation results, errors, and evaluation results are communicated clearly through the web interface.

Thus, the detailed design provides a direct bridge between the **architecture defined in Section 3**, the **requirements established in the SRS**, and the **testing and integration flows established in the STP**. This provides the foundation for the UML sequence diagrams, API definitions, error handling, logging, monitoring, and UX design presented in the remaining sections of the SAD.

4.2 UML Sequence Diagrams



4.3 API Design

The Web Based ML Model Evaluator uses application interfaces to support communication between the web interface and the backend services. The API design follows the layered modular architecture defined in Section 3 and supports the major project workflow from dataset processing to machine-learning model evaluation.

The frontend communicates with the Django backend using HTTP/HTTPS-based requests. JSON is used for structured API data where applicable, while dataset files are transferred using multipart file-upload mechanisms. All browser-to-application communication is protected using HTTPS with TLS 1.2 or higher. The APIs also enforce authentication and authorization so that users can access only their authorized projects, datasets, model runs, and evaluation results.

The following two component interfaces are defined as representative APIs for the system.

4.3.1 Dataset Management API

Component: Dataset Management Service

Purpose: Handles dataset upload, validation, metadata storage, and dataset preview.

Property	Definition
Endpoint	/api/projects/{project_id}/datasets/
Method	POST
Content Type	multipart/form-data
Authentication	Required
Input	CSV dataset file
Processing	File validation, size/format checking, dataset structure validation and metadata extraction
Output	Dataset identifier, validation status, metadata and preview information
Storage	File Storage + Relational Database

Request

POST /api/projects/{project_id}/datasets/

Content-Type: multipart/form-data

file: dataset.csv

Successful Response

```
{
  "status": "success",
  "dataset_id": "DS001",
  "filename": "dataset.csv",
  "rows": 1500,
  "columns": 12,
  "validation": "passed",
  "preview_available": true
}
```

Error Responses

Status	Error	Description
400	Invalid dataset	Dataset structure or contents are invalid
401	Unauthorized	User authentication is required
403	Forbidden	User does not have access to the selected project
413	File too large	Dataset exceeds the permitted size
415	Unsupported file type	Uploaded file is not a supported format
500	Processing error	Unexpected server-side processing failure

Interface Flow: Web UI → API Layer → Dataset Management Service → File Storage / Database → API Layer → Web UI

This interface corresponds to the SRS requirements for dataset upload, validation, preview, and dataset-related processing. The SRS specifically requires validation of uploaded datasets and defines multipart file upload for dataset transfer.

4.3.2 Model Evaluation API

Component: Evaluation Service

Purpose: Handles model evaluation using the trained model and held-out test data and returns task-specific evaluation metrics.

Property	Definition
Endpoint	/api/projects/{project_id}/evaluation/
Method	POST
Content Type	application/json
Authentication	Required
Input	Model run identifier and evaluation configuration
Processing	Retrieve trained model, execute testing using held-out data and calculate appropriate metrics
Output	Evaluation status and task-specific performance metrics
Storage	Relational Database + File Storage where required

Request

```
{
  "model_run_id": "RUN001",
  "task_type": "classification"
}
```

Successful Classification Response

```
{
  "status": "completed",
  "model_run_id": "RUN001",
  "task_type": "classification",
  "metrics": {
    "accuracy": 0.92,
    "precision": 0.91,
    "recall": 0.90,
    "f1_score": 0.905
  }
}
```

Successful Regression Response

```
{
  "status": "completed",
  "model_run_id": "RUN002",
  "task_type": "regression",
  "metrics": {
    "mae": 2.41,
    "rmse": 3.17,
    "r2": 0.87
  }
}
```

Error Responses

Status	Error	Description
400	Invalid evaluation request	Missing or invalid evaluation parameters
401	Unauthorized	User authentication is required
403	Forbidden	User does not have access to the model run
404	Model run not found	Requested trained model/run does not exist
409	Evaluation unavailable	Required training or preprocessing stage is incomplete
500	Evaluation error	Model testing or metric calculation failed

Interface Flow: Web UI → API Layer → Evaluation Service → File Storage / ML Library → Evaluation Service → Database → API Layer → Web UI

The evaluation interface follows the SRS requirement that trained models are tested using previously unseen test data and that appropriate classification or regression metrics are calculated.

4.3.3 API Design Principles

The APIs follow the following design principles:

1. **Secure communication:** All browser-to-backend API communication uses HTTPS with TLS 1.2 or higher.
2. **Authentication and authorization:** Protected APIs require authenticated access and project/resource ownership checks.
3. **Structured data exchange:** JSON is used for structured request and response data where applicable.
4. **Multipart file upload:** Dataset files are transferred using multipart upload.
5. **Validation:** Uploaded files, user inputs, model parameters, and evaluation requests are validated before processing.
6. **Consistent error handling:** APIs return appropriate error information without exposing sensitive internal system details.
7. **Processing status:** Longer-running training or evaluation operations provide processing-status information to the user.
8. **Modular interfaces:** APIs provide defined boundaries between the web interface and backend services.

These principles are directly aligned with the communication and software-interface requirements specified in the SRS.

4.4 Error Handling, Logging & Monitoring

The Web Based ML Model Evaluator shall provide consistent error handling, logging, and monitoring mechanisms to maintain reliability, usability, security, and data integrity. Errors occurring during dataset validation, preprocessing, model training, testing, evaluation, API communication, or report generation shall be handled without exposing internal system details to the user.

The design separates **user-facing error messages** from **internal diagnostic information**. Users receive clear and understandable messages indicating the problem and, where applicable, the corrective action. Internal logs contain sufficient information for debugging and maintenance while excluding sensitive information such as passwords, authentication tokens, and other confidential data. This approach is consistent with the error-handling and security requirements defined in the SRS and STP. Team-13_Web-Based-ML-Model-Eval...

4.4.1 Error Handling

The system shall handle errors at the appropriate application layer and return a controlled response to the user interface.

The major error conditions include:

Error Category	Example Condition	System Response
Authentication Error	Invalid login credentials	Display authentication failure message
Authorization Error	User attempts to access another user's project	Deny access and display an appropriate message
Dataset Upload Error	Unsupported, corrupted, or oversized file	Reject upload and display validation message
Dataset Processing Error	Invalid dataset structure or preprocessing failure	Stop processing and display error
Model Configuration Error	Incompatible model/task or invalid parameters	Reject configuration and request correction
Training Error	Model training fails	Mark training operation as failed and display status
Testing/Evaluation Error	Evaluation cannot be completed	Display evaluation failure and preserve diagnostic information
API/Communication Error	Backend request fails	Return controlled error response
Report Generation Error	Report cannot be generated	Display report-generation failure
Storage Error	Required project/result information cannot be stored	Prevent incomplete operation from being treated as successful

The system shall ensure that errors do not expose internal implementation details, sensitive information, passwords, tokens, or other protected data. The SRS specifically requires clear error messages for dataset validation, preprocessing, training, and evaluation failures.

4.4.2 Error Response Structure

For API-based operations, errors shall be returned in a structured format so that the Web UI can display an understandable message.

Example:

```
{
  "status": "error",
  "error_code": "INVALID_DATASET",
  "message": "The uploaded dataset could not be processed.",
  "details": "Please verify the file format and dataset structure."
}
```

The detailed internal exception information shall not be exposed directly to the user. Internal diagnostic information may instead be recorded in the application logs for authorized developers and administrators.

4.4.3 Logging

The system shall maintain application and security logs for important system events. Logging shall support debugging, maintenance, security analysis, and verification of system behaviour.

Important events to be logged include:

- User login attempts and authentication events.
- Access attempts to protected resources.
- Dataset upload and validation events.
- Preprocessing operations.
- Model training and model-run events.
- Model testing and evaluation failures.
- API errors and communication failures.
- Report-generation events.
- Administrative actions.
- Important system or service failures.

The logs shall contain sufficient information to understand the occurrence and context of an event while protecting sensitive information.

Sensitive Information Protection

The following information shall **not** be recorded in application or security logs:

- User passwords.
- Authentication tokens.
- Sensitive credentials.
- Other confidential information that is not required for debugging or auditing.

This follows the security-logging requirements established in the project architecture.

4.4.4 Processing Status Management

Long-running operations such as model training, evaluation, and report generation shall provide clear processing-status information to the user.

The system shall represent the major processing states as: **Pending → Running → Completed**

or, when an operation fails: **Pending → Running → Failed**

The Web UI shall display the current state so that users understand whether an operation is waiting, currently being processed, successfully completed, or unsuccessful.

The STP specifically identifies **training-status handling**, processing failures, and appropriate status information for longer-running operations as areas requiring verification.

4.4.5 Monitoring

The system shall monitor important application activities and operational conditions to identify failures and performance issues.

The primary monitoring areas are:

Monitoring Area	Purpose
API Errors	Identify failed or abnormal application requests
Model Training Status	Track pending, running, completed, and failed training operations
Evaluation Failures	Identify unsuccessful model-testing or evaluation operations
Dataset Processing	Identify validation and preprocessing failures
Report Generation	Track successful and failed report-generation operations
Service Availability	Monitor whether the application remains available
Security Events	Observe authentication and unauthorized-access events
System Performance	Verify application and dataset-preview response requirements

The monitoring design supports the SRS reliability and performance requirements. The SRS specifies that commonly used pages should meet the defined response-time target, dataset previews for supported datasets up to 10 MB should meet their response-time target, and the system should maintain at least **99% scheduled availability excluding planned maintenance**.

4.4.6 Error Handling and Monitoring Flow

The overall approach can be summarized as:

User Request



Input / File Validation



Process Request



Success → Store Result → Update Status → Display Result

If an error occurs:

Process Request



Detect Error



Record Diagnostic Event



Protect Sensitive Information



Update Operation Status to Failed



Return Clear Error Message to User

This ensures that errors are handled in a controlled manner while providing developers and administrators with sufficient information for debugging and maintenance.

4.4 Summary

The error-handling, logging, and monitoring design ensures that the Web Based ML Model Evaluator provides **clear user feedback, controlled failure handling, secure logging, processing-status visibility, and operational monitoring**. These mechanisms support the SRS requirements for usability, reliability, security, data integrity, and maintainability and the STP requirements for functional, security, performance, and reliability testing.

4.5 UX Design

The User Experience (UX) design of the Web Based ML Model Evaluator is intended to provide a clear, responsive, accessible, and understandable workflow for users performing machine-learning model evaluation. The UX follows the complete workflow defined in the SRS, allowing users to move from project creation and dataset upload through preprocessing, model selection, training, testing, evaluation, comparison, and report generation.

The interface is designed for users with basic or intermediate knowledge of machine learning and therefore provides clear labels, structured forms, understandable status information, and meaningful error messages throughout the workflow.

4.5.1 User Interface Structure

The system provides the following major user interfaces:

Interface	UX Purpose
Login and Registration	Allows users to securely create accounts and access the application.
Dashboard	Displays evaluation projects and provides project-management options.
Dataset Upload	Allows users to upload datasets and view file-validation status.
Dataset Preview	Displays dataset rows, columns, data types, and basic dataset information.
Data Preprocessing	Allows users to configure target selection, task type, missing-value handling, encoding, scaling, and train-test split.
Model Selection	Allows users to select a suitable classification or regression model and configure parameters.

Training and Testing	Displays training/testing progress and processing status.
Results Dashboard	Displays evaluation metrics, tables, comparisons, and charts.
Report Interface	Allows users to generate and download evaluation reports.

These interfaces are directly based on the user-interface requirements specified in the SRS.

4.5.2 Workflow-Oriented Navigation

The UX follows a structured workflow so that users can complete the model-evaluation process in a logical order:

Login



Dashboard / Create Project



Upload Dataset



Validate & Preview Dataset



Preprocess Dataset



Select ML Task & Model



Train Model



Test Model



Evaluate Performance



Compare & Visualize Results



Generate Report

This navigation structure is consistent with the end-to-end workflow specified in the STP.

The interface should provide clear navigation between these stages while preventing users from performing dependent operations before the required previous stage has been completed.

4.5.3 Responsive Design

The web interface shall be responsive and usable across:

- Desktop computers
- Laptop computers
- Tablet devices

The layout shall adapt to different screen sizes while maintaining readable forms, tables, navigation elements, charts, and status information.

The SRS explicitly requires a responsive web-based interface and identifies desktop, laptop, and tablet environments as supported client environments.

4.5.4 Clear Interaction and Feedback

The interface shall provide clear:

- Labels
- Forms
- Buttons
- Navigation elements
- Instructions
- Validation messages
- Processing-status information

User actions such as dataset upload, preprocessing, model training, testing, and report generation shall provide appropriate feedback so that the user can understand the current state of the operation.

For longer-running operations, the interface shall communicate whether the operation is: **Pending → Running → Completed** or **Pending → Running → Failed**

This is consistent with the project's requirement for processing-status information during longer operations.

4.5.5 Dataset and Result Presentation

The UX shall provide readable data representations for the two major information-heavy areas of the system.

Dataset Presentation

The Dataset Preview interface shall clearly present:

- Dataset rows
- Column names
- Data types
- Basic dataset information

This allows the user to verify the uploaded dataset before proceeding with preprocessing.

Model Results Presentation

The Results Dashboard shall present:

- Evaluation metrics
- Model comparison tables
- Graphical visualizations
- Preferred-model information
- Previously stored evaluation results where applicable

The SRS requires side-by-side comparison of selected models and graphical visualization of model-performance metrics.

4.5.6 Accessibility

The UX shall support accessibility requirements for the major application screens.

The design shall include:

- **Keyboard navigation** for applicable interface elements.
- **Descriptive labels** for forms and controls.
- **Visible focus indicators** to identify the currently focused element.
- **Sufficient colour contrast** for text and important status indicators.
- Clear and understandable messages that do not rely only on colour to communicate important information.

These requirements are consistent with the accessibility requirements defined in the SRS and the usability/accessibility testing planned in the STP.

4.5.7 Error and Validation Feedback

The UX shall provide **clear and actionable error messages** when an operation cannot be completed.

Examples include:

- Invalid or unsupported dataset upload.
- Dataset validation failure.
- Invalid preprocessing configuration.
- Invalid model parameters.
- Model-training failure.
- Evaluation failure.
- Report-generation failure.

Error messages should explain the problem in understandable terms and, where possible, indicate what the user needs to correct.

Internal system details, passwords, tokens, or other sensitive information shall not be displayed to the user.

This follows the SRS requirement that processing failures provide understandable messages without exposing internal system details.

4.5.8 Confirmation for Destructive Actions

Destructive operations such as **project deletion** shall require explicit user confirmation before execution. This prevents accidental deletion of evaluation projects and their associated information.

4.5.9 UX Design Principles

The overall UX design follows these principles:

Principle	Application in the System
Simplicity	Keep the ML evaluation workflow understandable and structured.
Consistency	Maintain consistent labels, navigation, forms, messages, and status indicators.
Feedback	Inform users about validation, processing, training, evaluation, and report status.
Accessibility	Support keyboard navigation, descriptive labels, focus indicators, and sufficient colour contrast.
Error Prevention	Validate datasets and user/model inputs before processing.
Error Recovery	Provide clear messages explaining failures and possible corrective actions.
Readability	Present datasets, metrics, tables, and charts in an understandable form.
Responsive Design	Support desktop, laptop, and tablet screens.

4.5.10 UX and Usability Verification

The UX design will be verified through the usability and accessibility testing activities defined in the STP.

The testing will verify:

- Clarity of labels, buttons, forms, and instructions.
- Understandability of validation and error messages.
- Consistency of the user interface.
- Keyboard navigation.
- Accessibility-related interface requirements.
- Correct display of processing-status information.
- Readability of dataset and model-comparison information.

The SRS identifies usability and accessibility as quality attributes, while the STP includes dedicated usability and accessibility testing.

Section Summary

The UX design provides a **responsive, workflow-oriented, accessible, and feedback-driven interface** for the complete Web Based ML Model Evaluator. It ensures that users can move systematically from **dataset upload** → **preprocessing** → **model selection** → **training** → **testing** → **evaluation** → **comparison** → **reporting**, while receiving clear status, validation, and error information throughout the process.

4.6 Open Issues & Next Steps

The following open issues and next steps identify the remaining activities required to complete, validate, and prepare the Web Based ML Model Evaluator for final implementation and release. These items are based on the current SAD design, the SRS requirements, and the testing activities defined in the STP.

4.6.1 Open Issues

No.	Open Issue	Impact / Consideration
1	Authentication and Session Policy	The final deployed authentication, session-management, and secret-management approach needs to be finalized before implementation and deployment.
2	Dataset Size Limits	Final dataset-size limits need to be established based on available system resources and performance testing.
3	Report Layout and Generation	The final report structure and report-generation library need to be finalized.
4	File and Artifact Retention	Retention and cleanup rules for uploaded datasets, trained model artifacts, and generated reports need to be defined.
5	Background Processing	Celery and Redis may need to be introduced if performance testing shows that model training or evaluation operations are sufficiently long-running to require asynchronous processing.
6	Security and Accessibility Validation	Security, accessibility, performance, regression, and acceptance testing must be completed before final release.

These open issues are consistent with the current SAD draft and the project's defined requirements and testing scope.

4.6.2 Next Steps

The planned next steps are:

1. Finalize Implementation Details

Complete the implementation of the architecture and interfaces defined in this SAD, including the Django backend, web interface, dataset-processing modules, model-evaluation modules, storage mechanisms, and defined APIs.

2. Complete Integration

Integrate the major modules according to the defined workflow: **Authentication → Project Management → Dataset Upload → Validation → Preview → Preprocessing → Model Selection → Training → Testing → Evaluation → Comparison → Visualization → Report Generation**

The STP identifies these interactions as important integration flows and defines the complete workflow for system testing.

3. Finalize Performance Configuration

Perform performance testing to establish suitable dataset-size limits and determine whether longer-running model operations require background processing using Celery and Redis.

4. Complete Security Validation

Verify HTTPS communication, authentication, authorization, password protection, file validation, input validation, and security logging as defined by the SRS and STP.

5. Complete Usability and Accessibility Testing

Verify the clarity of labels, forms, buttons, navigation, error messages, keyboard navigation, colour contrast, and status information.

6. Complete Functional and Integration Testing

Execute the functional and integration test cases covering dataset processing, preprocessing, model selection, training, testing, evaluation, result comparison, visualization, and report generation.

7. Complete Acceptance Testing

Verify that the implemented system satisfies the SRS acceptance criteria and that the RTM provides traceability between requirements, design specifications, system modules, and test cases.

8. Finalize Release Configuration

After successful testing, finalize deployment configuration, documentation, data-retention policies, report generation, and operational monitoring before the final release.

4.6.3 Expected Outcome

Completion of these activities will provide a validated implementation of the architecture and design specified in this SAD. The final system should support the complete workflow from **dataset submission to model evaluation and reporting**, while satisfying the functional, performance, usability, security, reliability, and data-integrity requirements established in the SRS and verified through the STP.

5. Appendices

5.1 Glossary

Term / Abbreviation	Definition
API	Application Programming Interface used for communication between software components.
CSV	Comma-Separated Values, a format for storing tabular datasets.
Dataset	Collection of data used for training and evaluating ML models.
Feature	Input variable used by a machine-learning model.
Target	Output variable that the model is trained to predict.
Classification	ML task that predicts categories or classes.
Regression	ML task that predicts numerical values.
Preprocessing	Preparing and transforming data before model training.
Training	Process of learning patterns from training data to build a model.
Testing	Evaluating a trained model using unseen test data.
Evaluation Metric	Measure used to assess model performance, such as accuracy, F1-score, MAE, RMSE, or R^2 .
ML	Machine Learning.
UI	User Interface through which users interact with the application.
SRS	Software Requirements Specification.
STP	Software Test Plan.
RTM	Requirements Traceability Matrix.
SAD	Software Architecture and Design Specification.
TLS	Transport Layer Security used to protect data during transmission.
Model Run	A single execution of model training and/or evaluation.
Artifact	File or output generated during training or evaluation, such as a trained model or report.
Audit Event	Recorded security or system activity, such as a login or access attempt.

These definitions are aligned with the terminology used in the project SRS and SAD.

5.2 References

- **Team 13 – Software Requirements Specification (SRS)**, Version 1.0, dated 06-09-2026.
- **Team 13 – Software Test Plan (STP)**, Version 1.1, dated 16-09-2026.
- **Team 13 – Requirements Traceability Matrix (RTM)**, included in the SRS.

- **College-provided Software Architecture and Design Specification (SAD) Template**, used as the structure and guideline for the Phase 3 SAD.
- **Team 13 – Software Architecture and Design Specification (SAD)**, Phase 3 project document, prepared based on the SRS, STP, RTM, and SAD template.

5.3 Tools

The following tools and technologies are used for developing, testing, documenting, and maintaining the **Web Based ML Model Evaluator**:

Tool / Technology	Purpose
Python	Backend development and machine-learning implementation.
Django	Web application backend and API development.
Pandas & NumPy	Dataset processing and numerical operations.
scikit-learn	Machine-learning model training and evaluation.
HTML/CSS/JavaScript & Bootstrap	Web interface development.
Chart.js / Plotly.js	Visualization of model evaluation results.
SQLite / PostgreSQL	Storage of users, projects, datasets, model runs, and evaluation results.
Git & GitHub	Version control and project management.
Mermaid UML Renderer	Creation of UML diagrams for system documentation.
Postman / API Testing Tool	Testing API requests and responses.
Browser Developer Tools	API inspection and frontend debugging.

The listed tools are based on the technology and tooling identified across the project architecture and supporting documents.

6. Approvals

The completed Software Architecture and Design Specification will be reviewed by each project team member and then submitted to the designated course coordinator for final approval.

Role	Name	Signature / Email	Date
Project Team Member	Pratham Kumar — PES2UG24CS368	prathamkumar041104@gmail.com	04-10-2026
Project Team Member	Nirupama Jayaraman — PES2UG24CS324	jnirupama06@gmail.com	04-10-2026
Project Team Member	Shreyas Bellad — PES2UG25CS824	shreyasbellad2006@gmail.com	04-10-2026
Project Team Member	Parinitha Acharya — PES2UG24CS339	parinitha.acharya2006@gmail.com	04-10-2026
Course Coordinator	Sheela Devi M.	sheelam@pes.edu	—

Project Contribution Details

S. No.	Name of the Student	SRN	Contribution
1	Parinitha Acharya	PES2UG24CS339	Section 1: Introduction – Purpose, Scope, Audience, and Definitions.
2	Nirupama Jayaraman	PES2UG24CS324	Sections 2–3: Document Overview and Architecture.
3	Pratham Kumar	PES2UG24CS368	Sections 3–4: Architecture and Design.
4	Shreyas Bellad	PES2UG25CS824	Section 5: Appendices – Glossary, References, and Tools.

THANK_YOU